BABU BANARASI DAS UNIVERSITY

LUCKNOW

SESSION:- 2025-26

SCHOOL OF COMPUTER APPLICATION

NoSQL Project On STUDENT MANAGEMENT SYSTEM USING MongoDB

Submitted by:

Name -: Shriya Shukla

Class -: BCADS 26

Roll no -: 1250258443

Submitted to:

Mr Harendra Rajpoot Sir

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers and the School of Computer Application for providing me with the opportunity to complete this NoSQL project on “Student Management System using MongoDB”. This project helped me understand how a document-oriented database stores, manages and retrieves student information.

I am thankful to my faculty members for their guidance, valuable suggestions and encouragement throughout the preparation of this project. I also appreciate the support of my classmates and everyone who helped me understand MongoDB commands and database operations.

Through this project, I gained practical knowledge of creating a database and collection, inserting multiple documents, applying comparison operators, updating records, deleting records and checking the number of stored documents. The project has strengthened my understanding of NoSQL concepts and their practical use in student data management.

INTRODUCTION

NoSQL databases are designed to handle data in flexible and scalable ways without depending on the fixed table structure commonly used in relational databases. MongoDB is a popular document-oriented NoSQL database that stores information in BSON documents, which are similar to JSON objects.

The purpose of this project is to develop a simple Student Management System using MongoDB. In this project, 50 student records are stored in a collection named students. Each document contains a student ID, name, age, gender, course, city and marks.

Different MongoDB operations are demonstrated to show how student information can be managed. These include creating/selecting a database, inserting 50 documents, finding records using comparison operators, logical operators, sorting, projection, limiting, updating, deleting, counting and pattern matching.

The commands are presented in MongoDB shell format and the corresponding output is shown as a screenshot-style result. This makes the project useful for understanding both the syntax and the practical result of each operation.

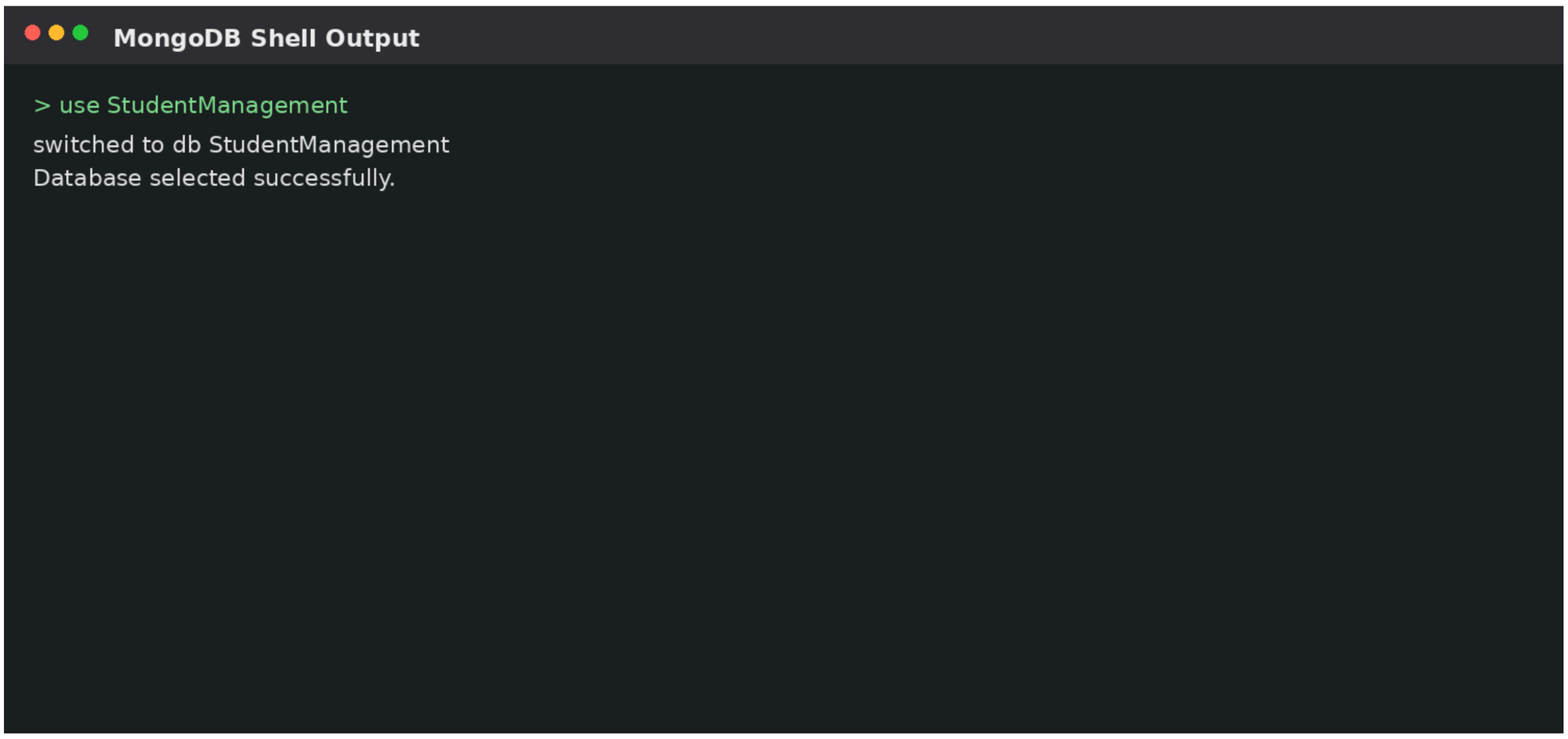
1. CREATE / SELECT DATABASE

The use command selects the StudentManagement database. If the database does not already exist, MongoDB creates it when data is first stored.

MongoDB Command

```
use StudentManagement
```

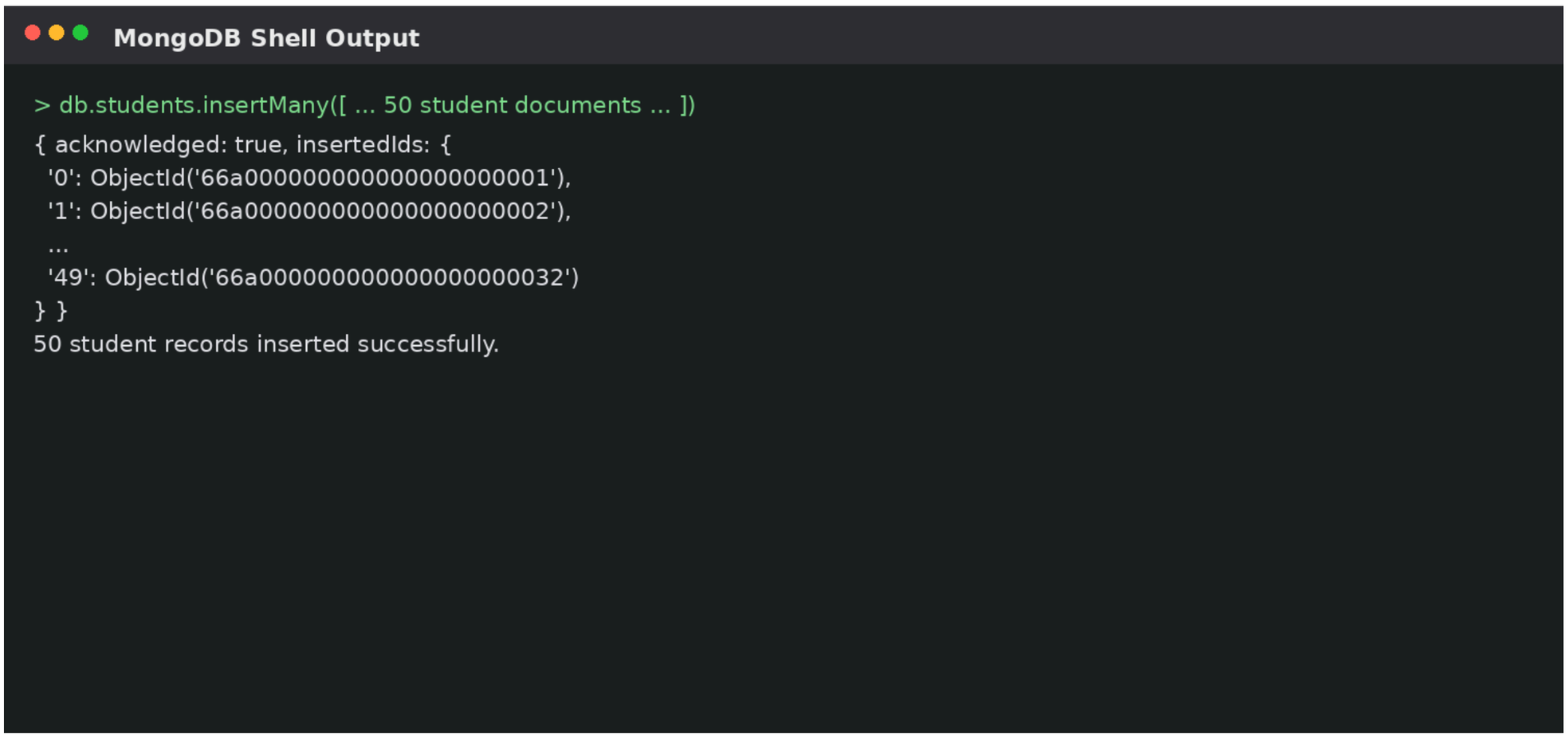
Output Screenshot



2. INSERT 50 STUDENT RECORDS

insertMany() adds multiple student documents to the students collection in a single operation. Here, exactly 50 student records are inserted.

Output Screenshot



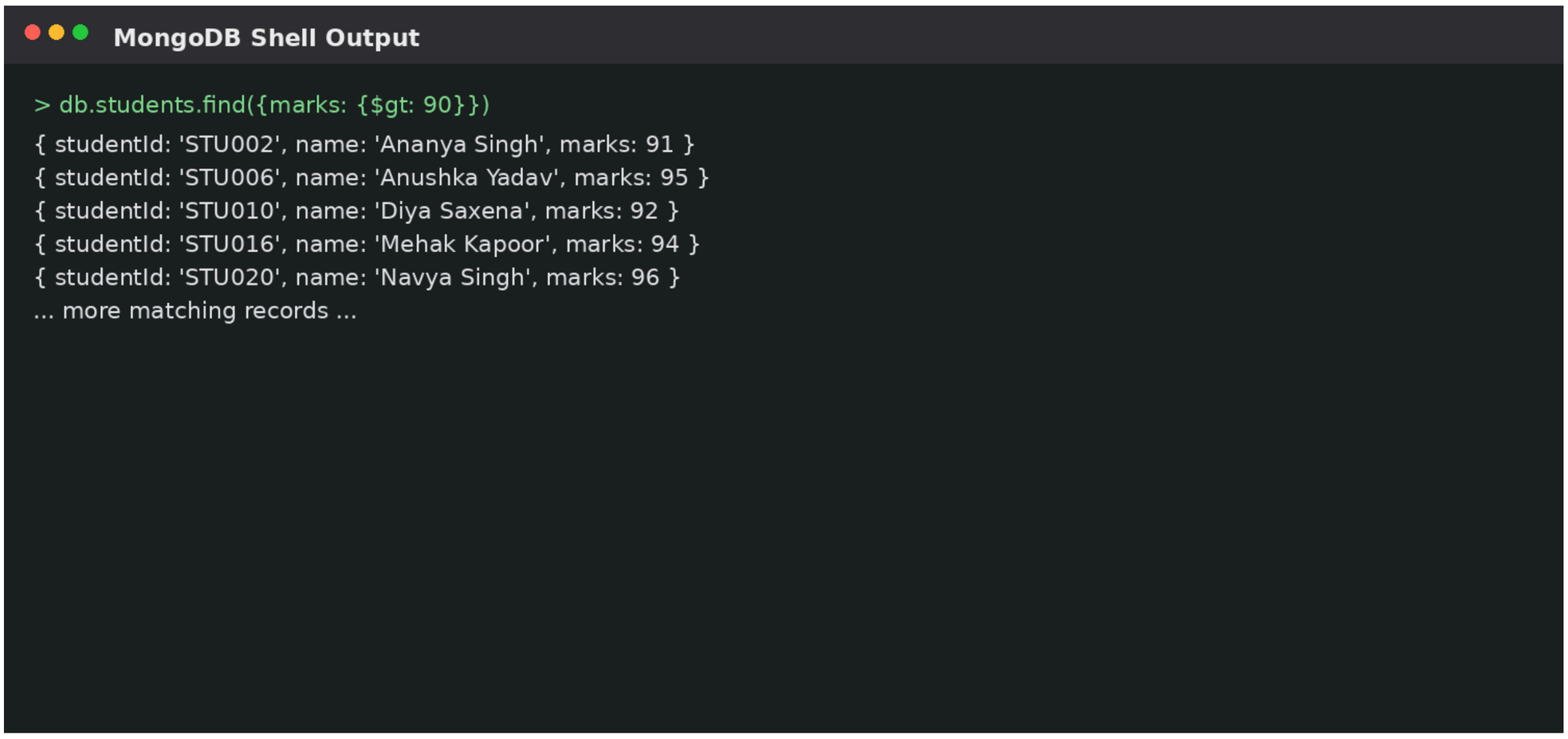
3. GREATER THAN ($gt)

$gt means “greater than”. This query displays students whose marks are greater than 90.

MongoDB Command

```
db.students.find({marks: {$gt: 90}})
```

Output Screenshot



4. LESS THAN ($lt)

$lt means “less than”. This query displays students whose marks are below 70.

MongoDB Command

```
db.students.find({marks: {$lt: 70}})
```

Output Screenshot

MongoDB Shell Output

```
> db.students.find({marks: {$lt: 70}})
{ studentId: 'STU005', name: 'Arjun Mishra', marks: 69 }
{ studentId: 'STU009', name: 'Dev Kumar', marks: 67 }
{ studentId: 'STU013', name: 'Kabir Khan', marks: 64 }
{ studentId: 'STU025', name: 'Rahul Gupta', marks: 68 }
{ studentId: 'STU031', name: 'Siddharth Jain', marks: 66 }
... more matching records ...
```

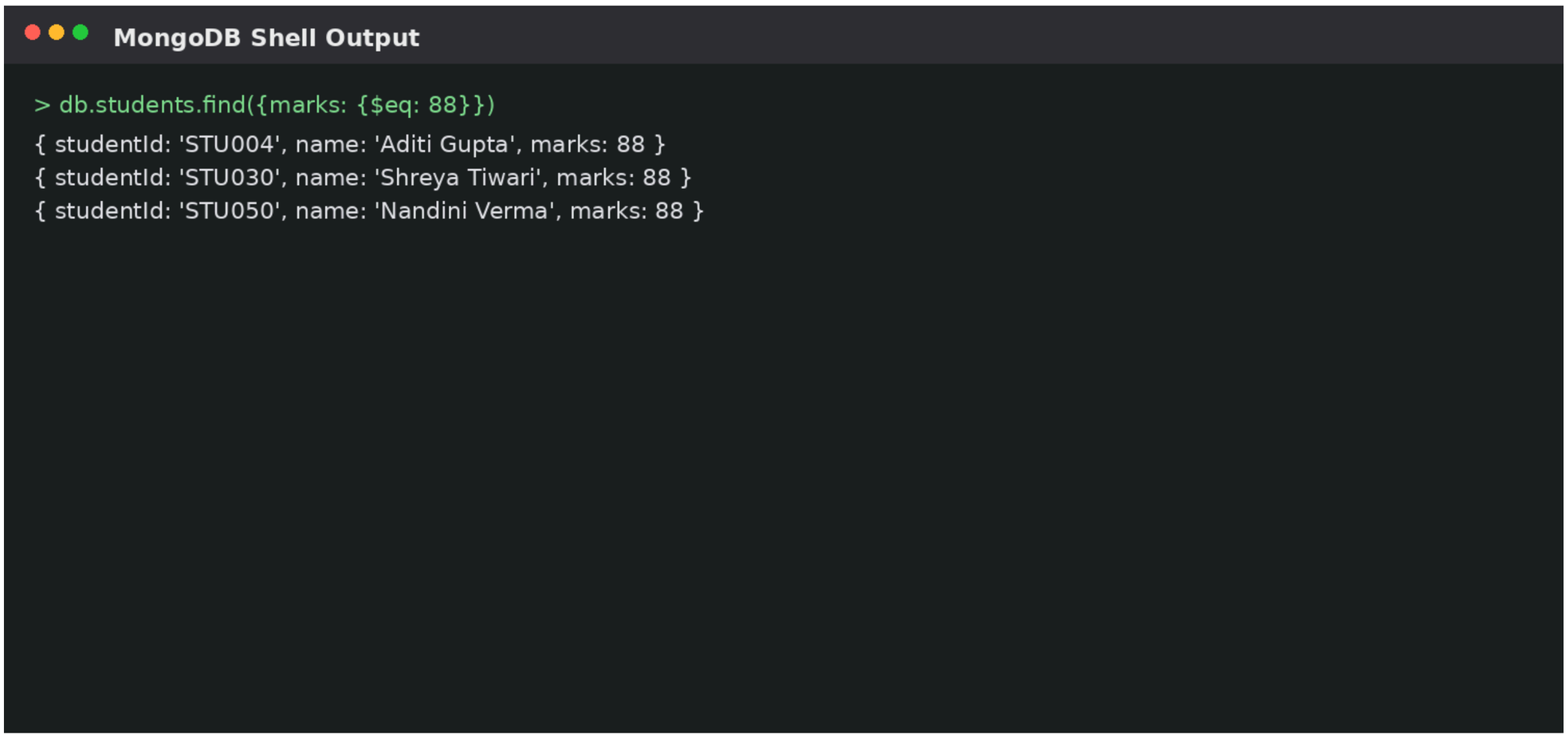
5. EQUAL TO ($eq)

$eq means “equal to”. This query returns all student documents whose marks are exactly 88.

MongoDB Command

```
db.students.find({marks: {$eq: 88}})
```

Output Screenshot



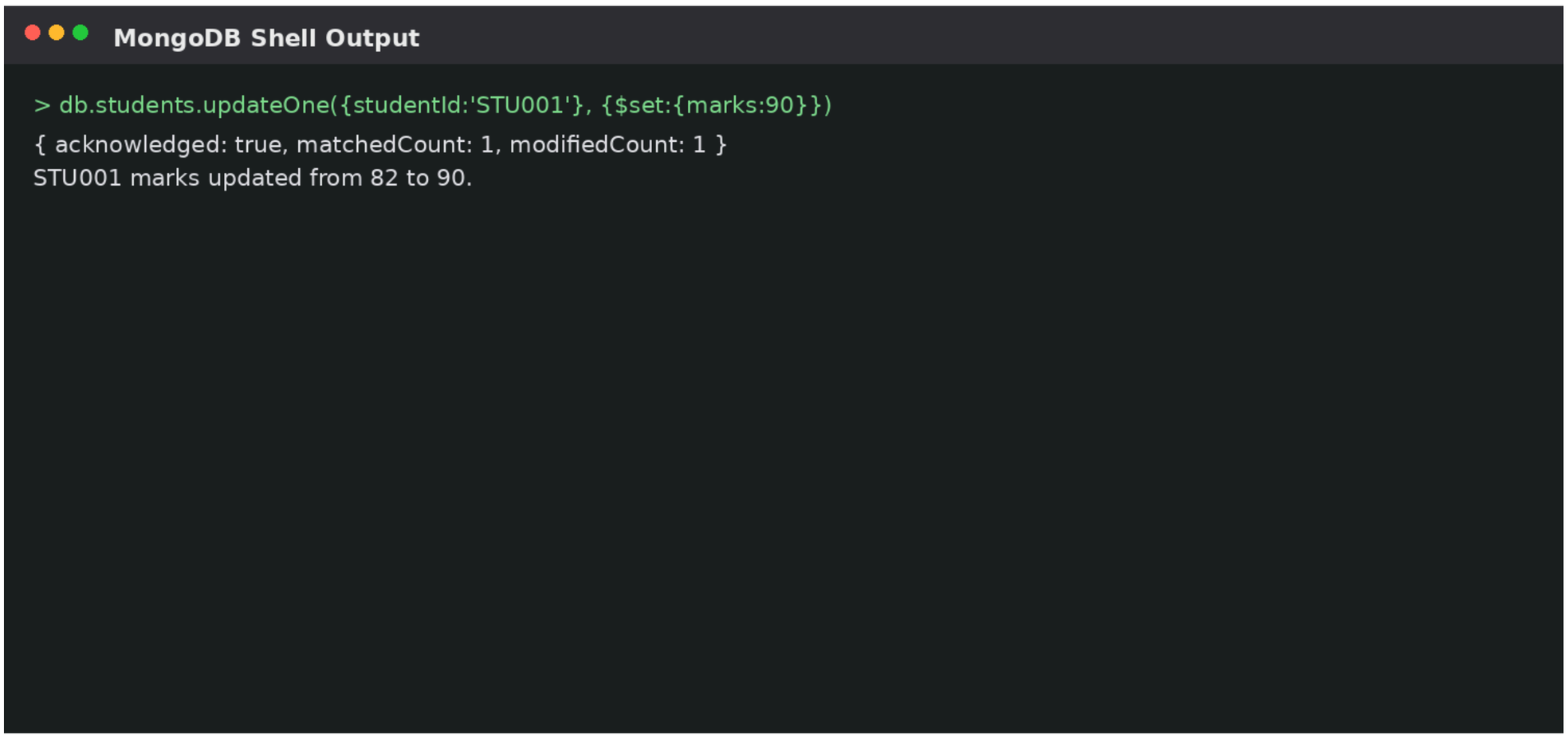
6. UPDATE OPERATION ($set)

updateOne() modifies the first document that matches the filter. The $set operator changes the selected field without replacing the complete document.

MongoDB Command

```
db.students.updateOne(
  {studentId: 'STU001'},
  {$set: {marks: 90}}
)
```

Output Screenshot



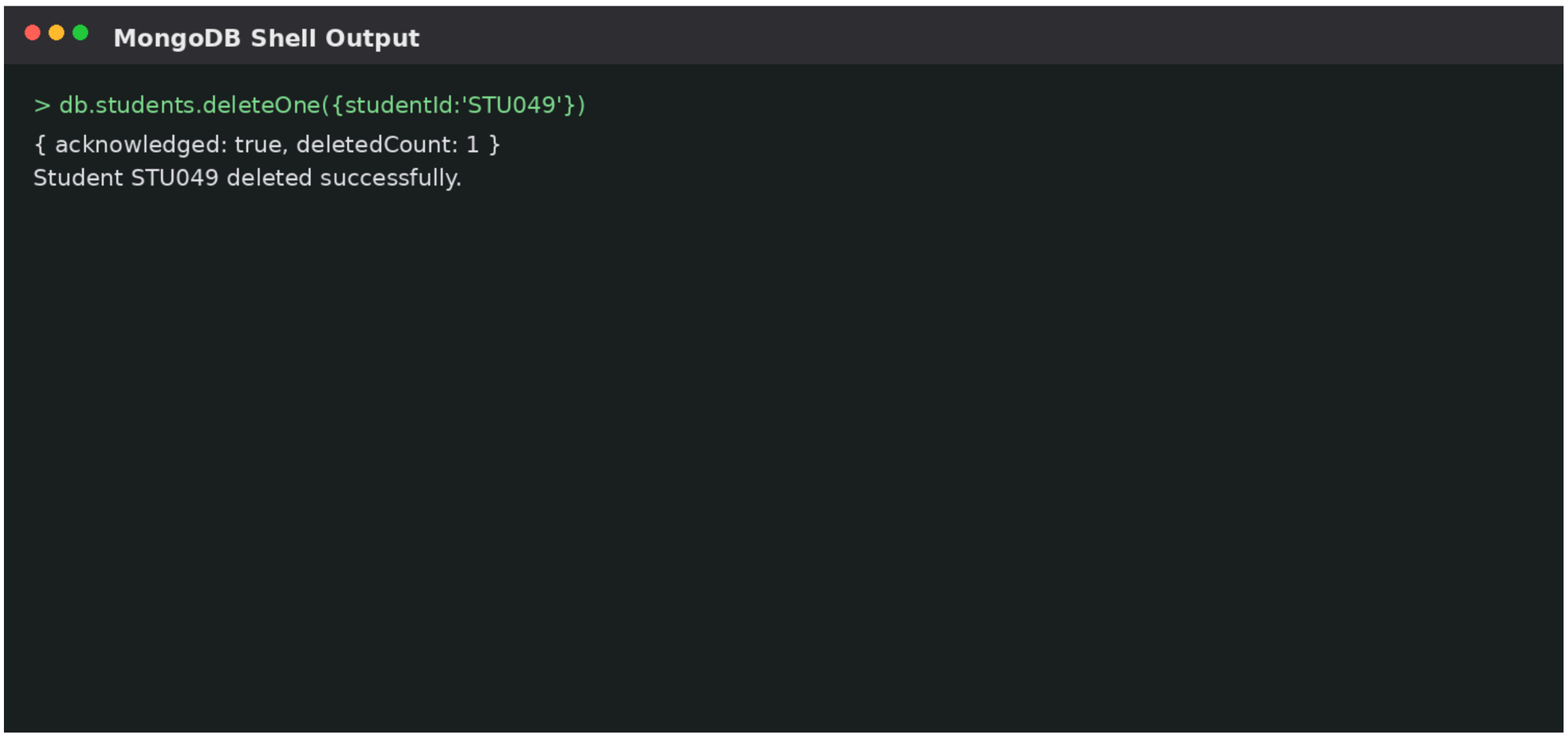
7. DELETE OPERATION

deleteOne() removes one document that matches the given condition. Here, the record with student ID STU049 is deleted.

MongoDB Command

```
db.students.deleteOne({studentId: 'STU049'})
```

Output Screenshot



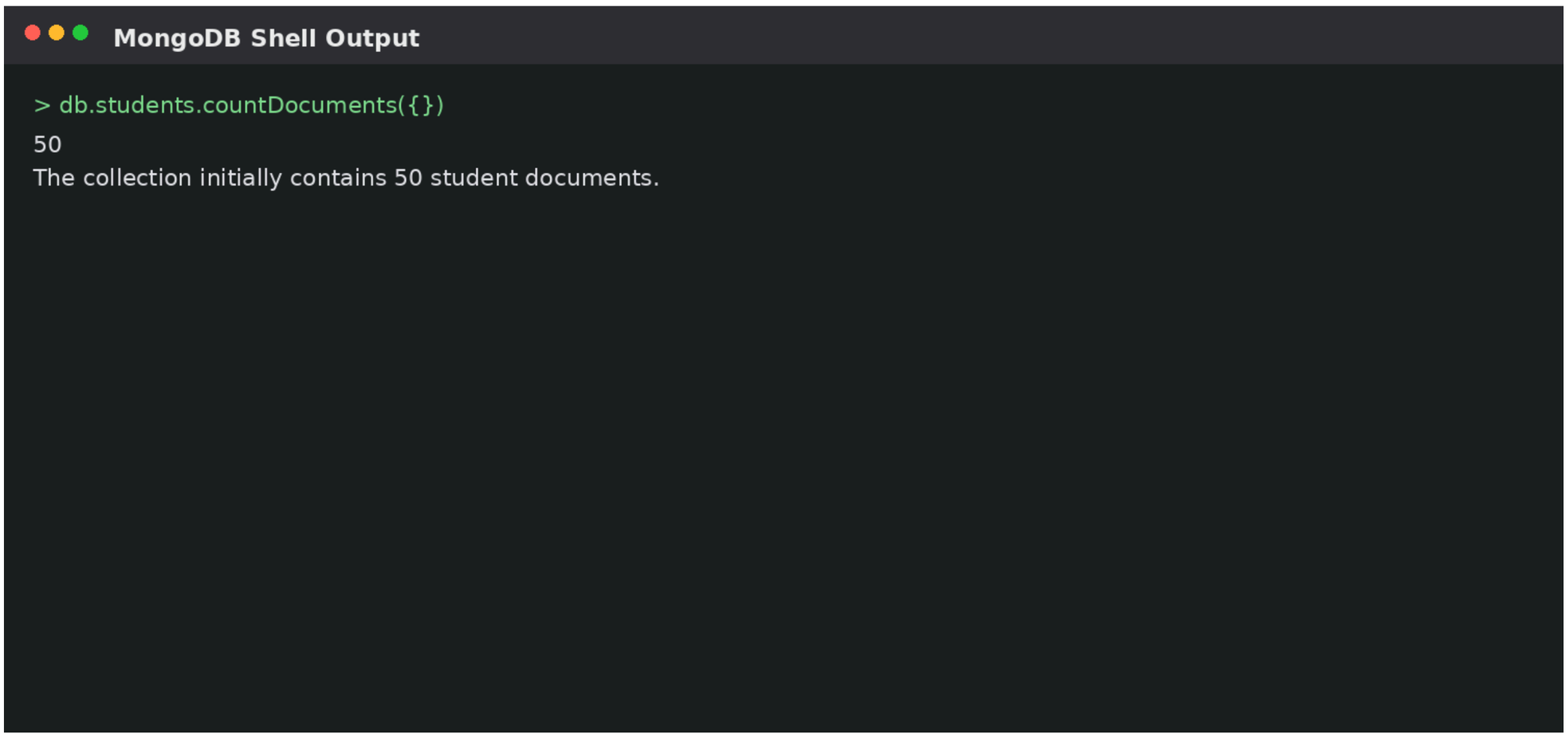
8. COUNT DOCUMENTS

countDocuments() returns the number of documents currently stored in the collection. Before the delete operation, the collection contains 50 records.

MongoDB Command

```
db.students.countDocuments({})
```

Output Screenshot



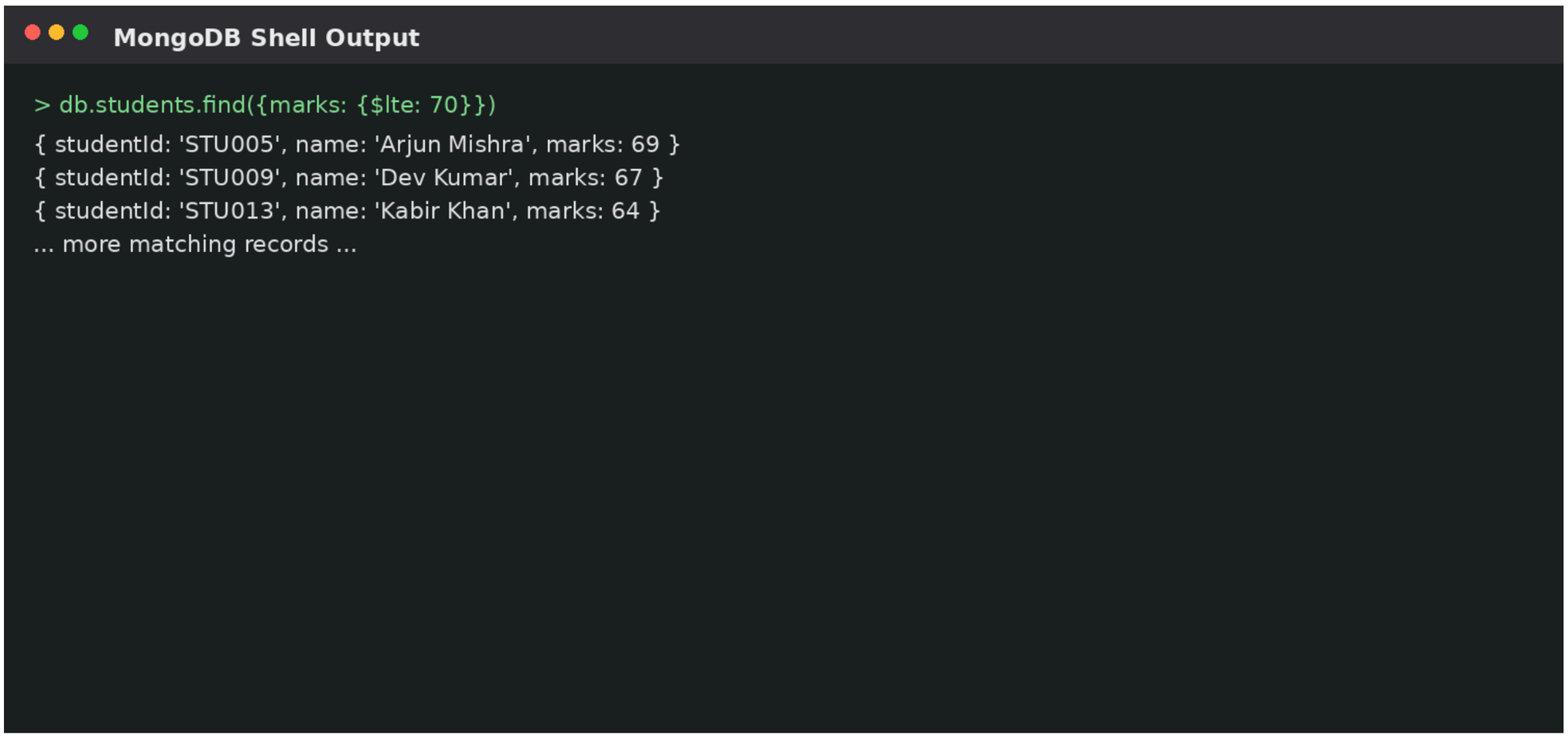
9. LESS THAN OR EQUAL TO ($lte)

$lte means “less than or equal to”. This query returns students whose marks are 70 or below.

MongoDB Command

```
db.students.find({marks: {$lte: 70}})
```

Output Screenshot



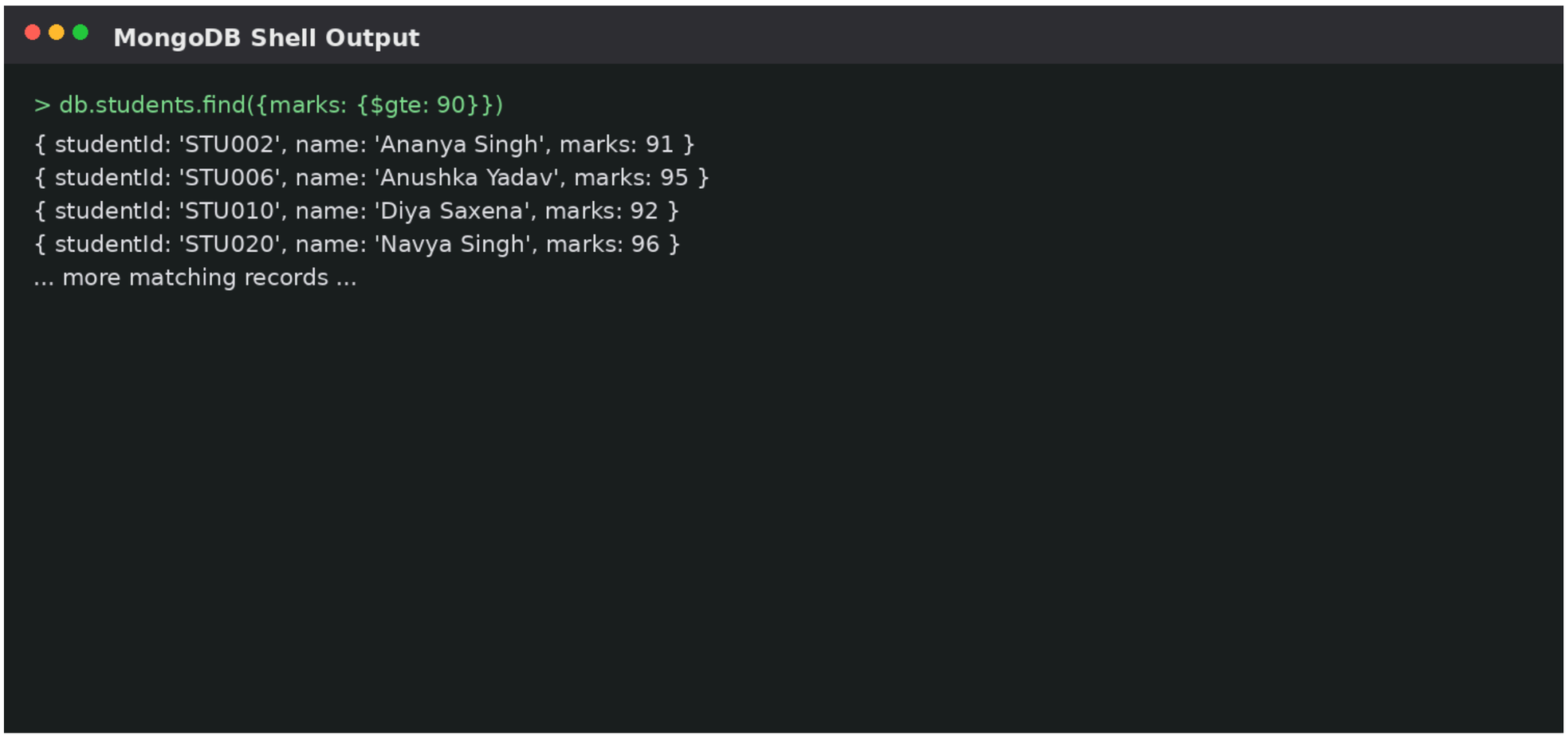
10. GREATER THAN OR EQUAL TO ($gte)

$gte means “greater than or equal to”. This query returns students whose marks are 90 or above.

MongoDB Command

```
db.students.find({marks: {$gte: 90}})
```

Output Screenshot



11. NOT EQUAL TO ($ne)

$ne means “not equal to”. This query returns documents where the marks value is different from 88.

MongoDB Command

```
db.students.find({marks: {$ne: 88}})
```

Output Screenshot



12. AND OPERATOR ($and)

$and requires all specified conditions to be true. Here, the query checks students older than 18 whose marks are at least 85.

MongoDB Command

```
db.students.find({$and: [{age: {$gt: 18}}, {marks: {$gte: 85}}]})
```

Output Screenshot



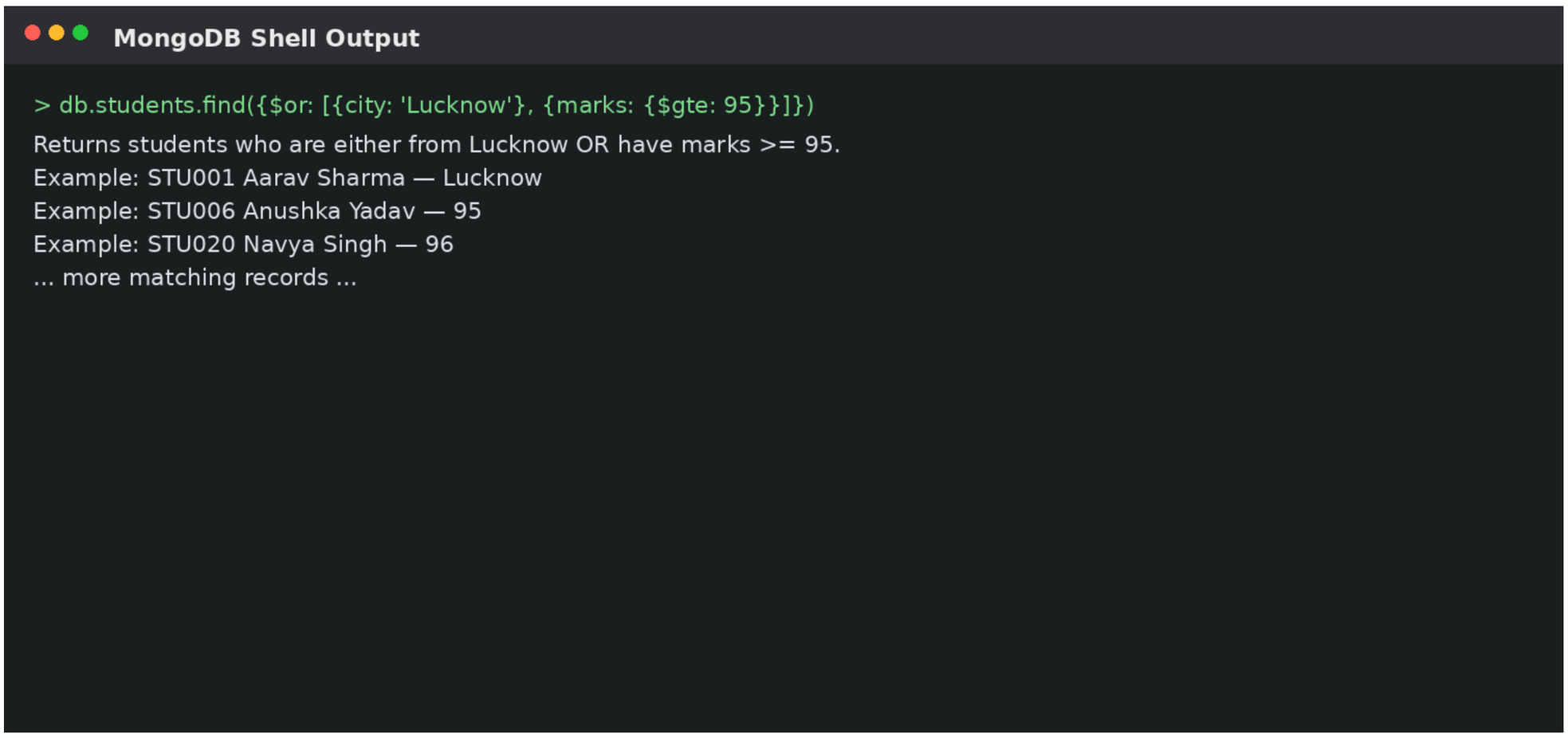
13. OR OPERATOR ($or)

$or returns documents satisfying at least one of the given conditions. Here, a student is returned if they are from Lucknow or have marks of 95 or above.

MongoDB Command

```
db.students.find({$or: [{city: 'Lucknow'}, {marks: {$gte: 95}}]})
```

Output Screenshot



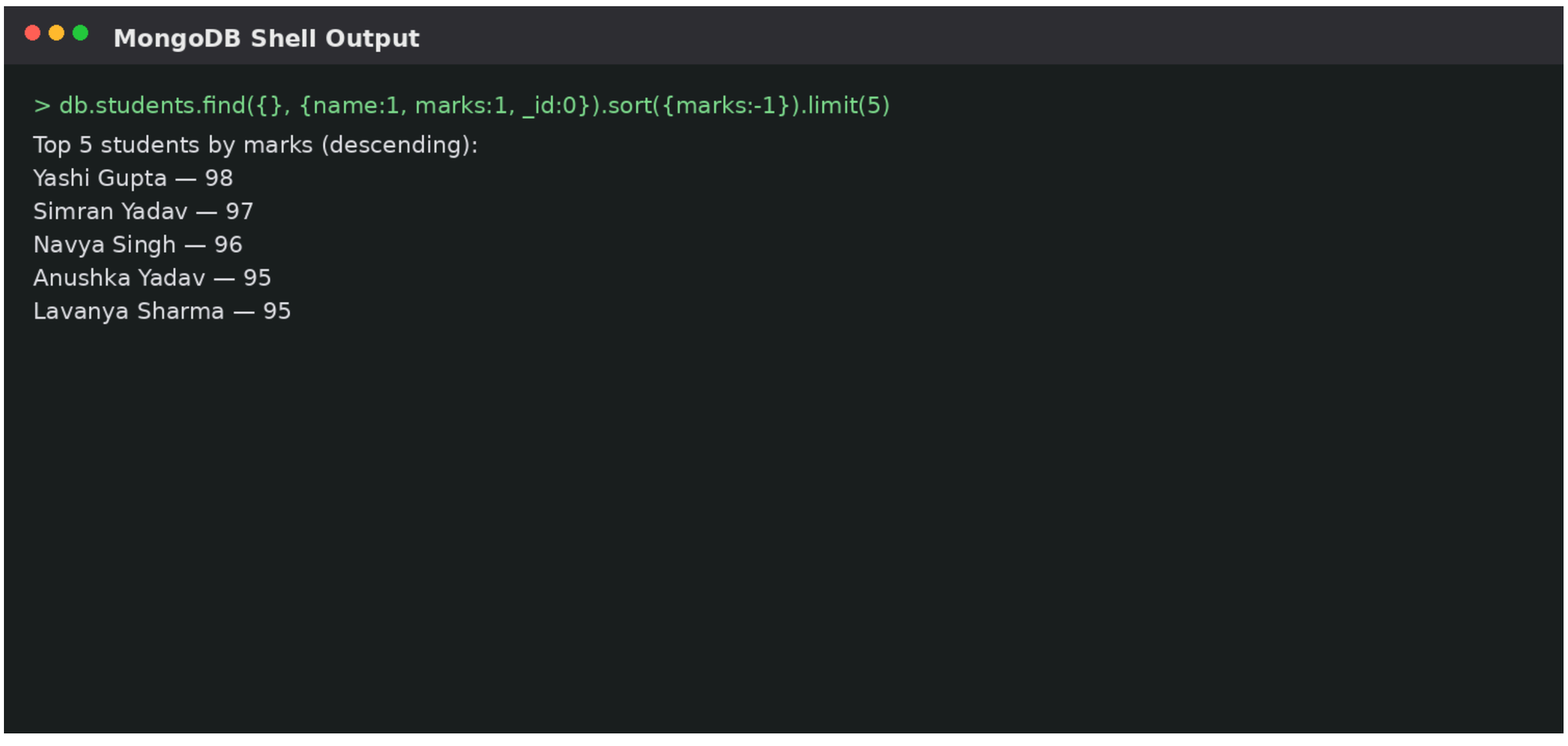
14. SORT OPERATION

sort() arranges query results. A value of -1 sorts in descending order and 1 sorts in ascending order. This command shows the five highest marks.

MongoDB Command

```
db.students.find({}, {name:1, marks:1, _id:0}).sort({marks:-1}).limit(5)
```

Output Screenshot



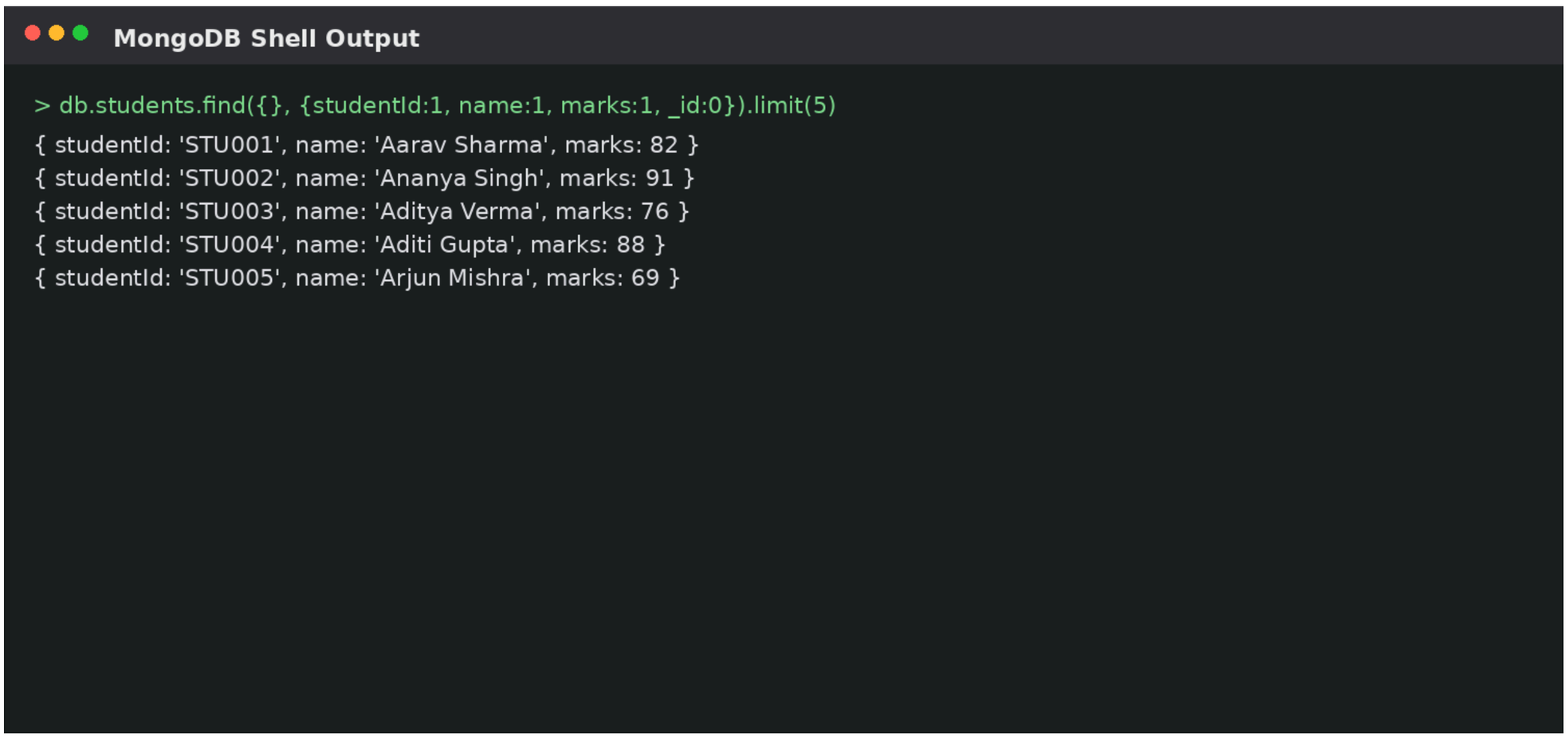
15. PROJECTION OPERATION

Projection selects only the required fields from each document. Here, only studentId, name and marks are displayed, while _id is hidden.

MongoDB Command

```
db.students.find({}, {studentId:1, name:1, marks:1, _id:0}).limit(5)
```

Output Screenshot



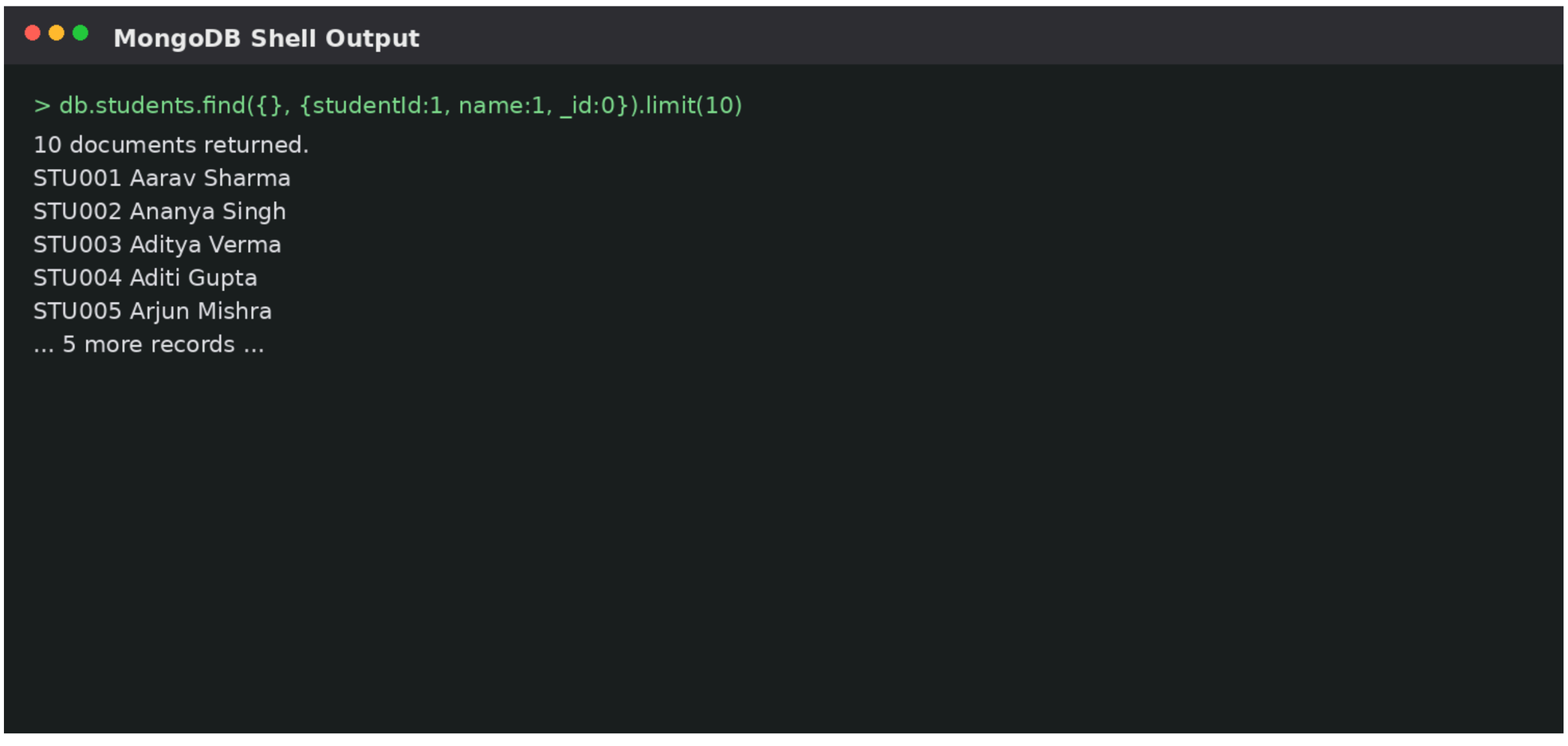
16. LIMIT OPERATION

limit() restricts the number of documents returned by a query. This command displays only the first 10 student records.

MongoDB Command

```
db.students.find({}, {studentId:1, name:1, _id:0}).limit(10)
```

Output Screenshot



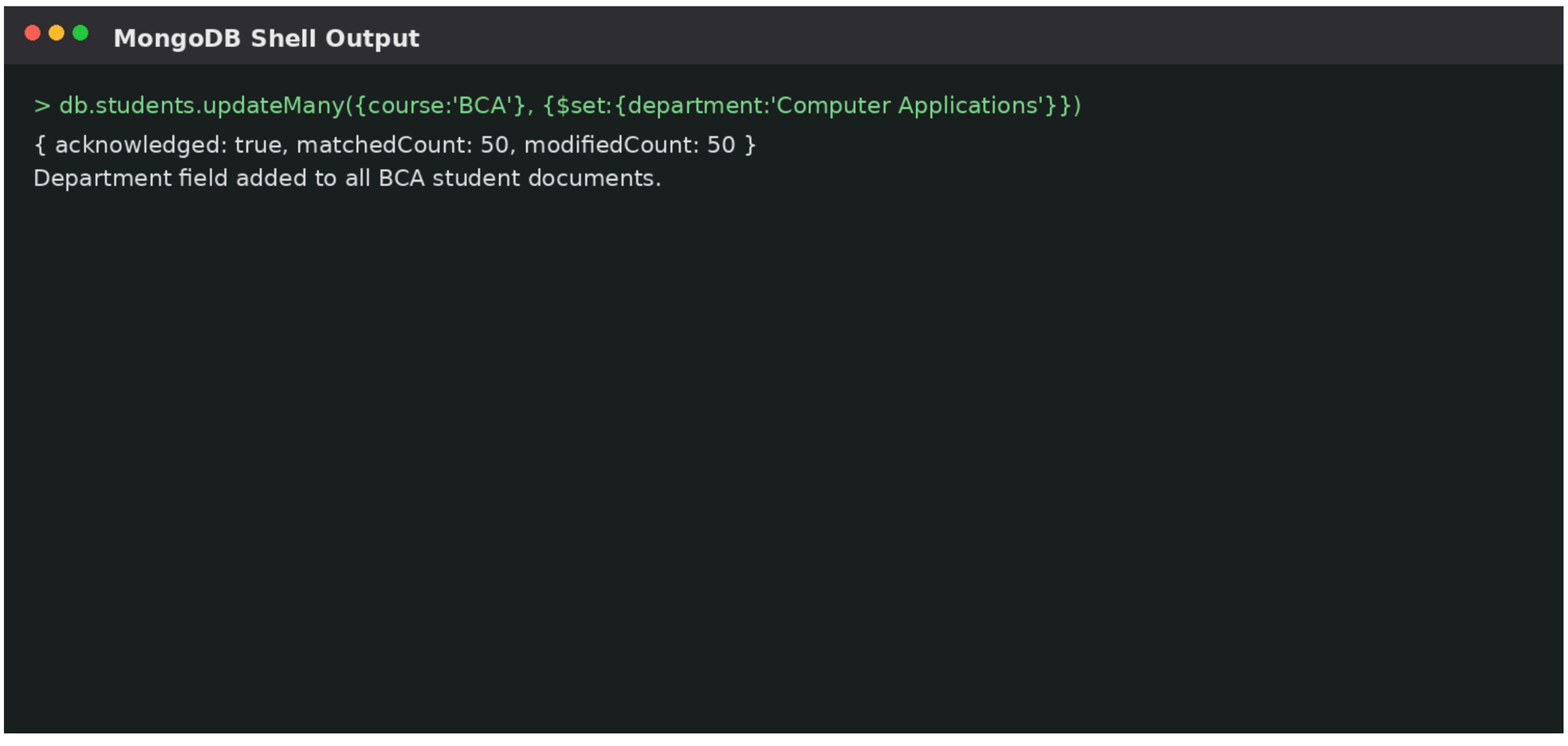
17. UPDATE MANY ($set)

updateMany() modifies every document that matches the condition. Here, the department field is added to all BCA student documents.

MongoDB Command

```
db.students.updateMany({course:'BCA'}, {$set:{department:'Computer Applications'}})
```

Output Screenshot



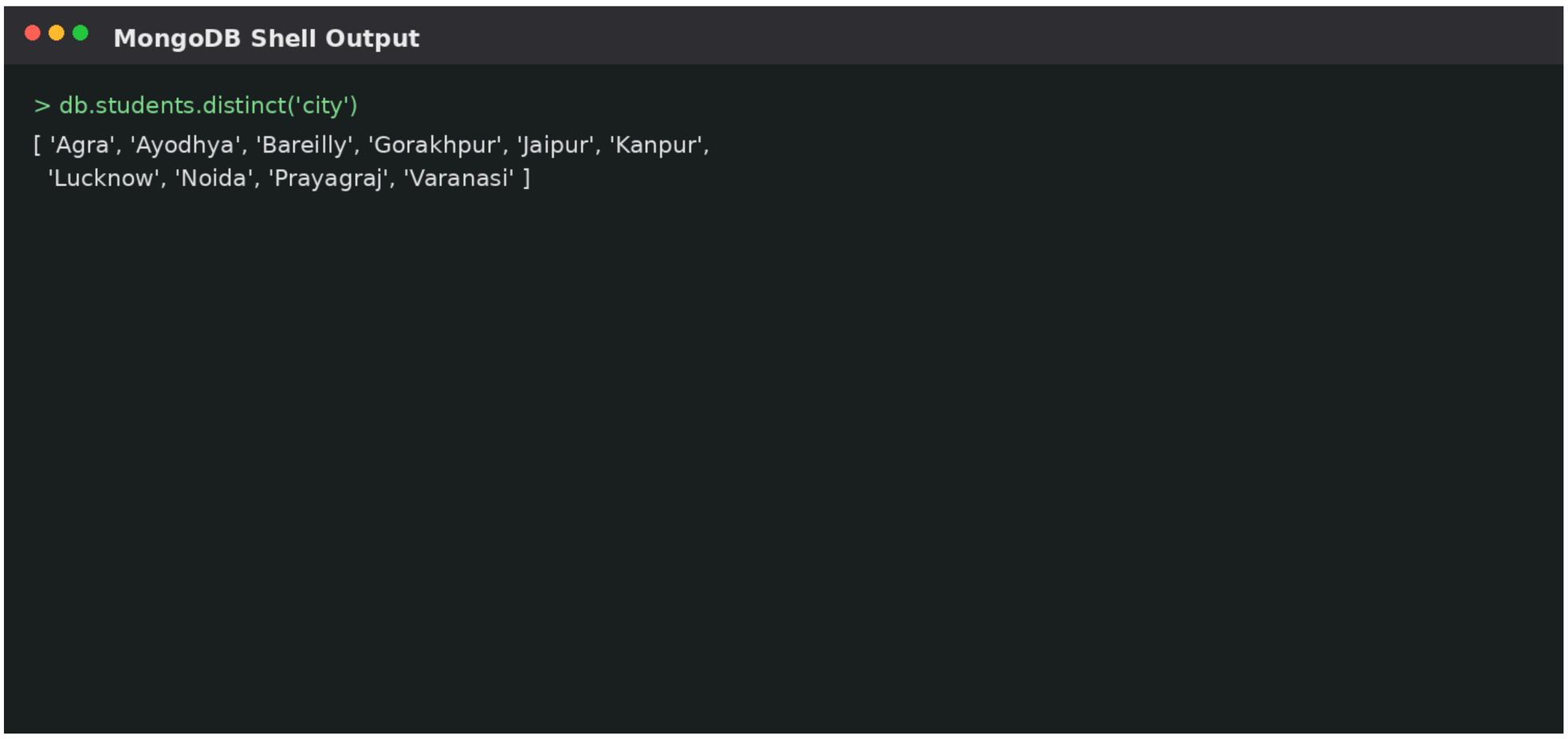
18. DISTINCT OPERATION

distinct() returns unique values of a specified field. This command lists each different city represented in the student collection without duplicates.

MongoDB Command

```
db.students.distinct('city')
```

Output Screenshot



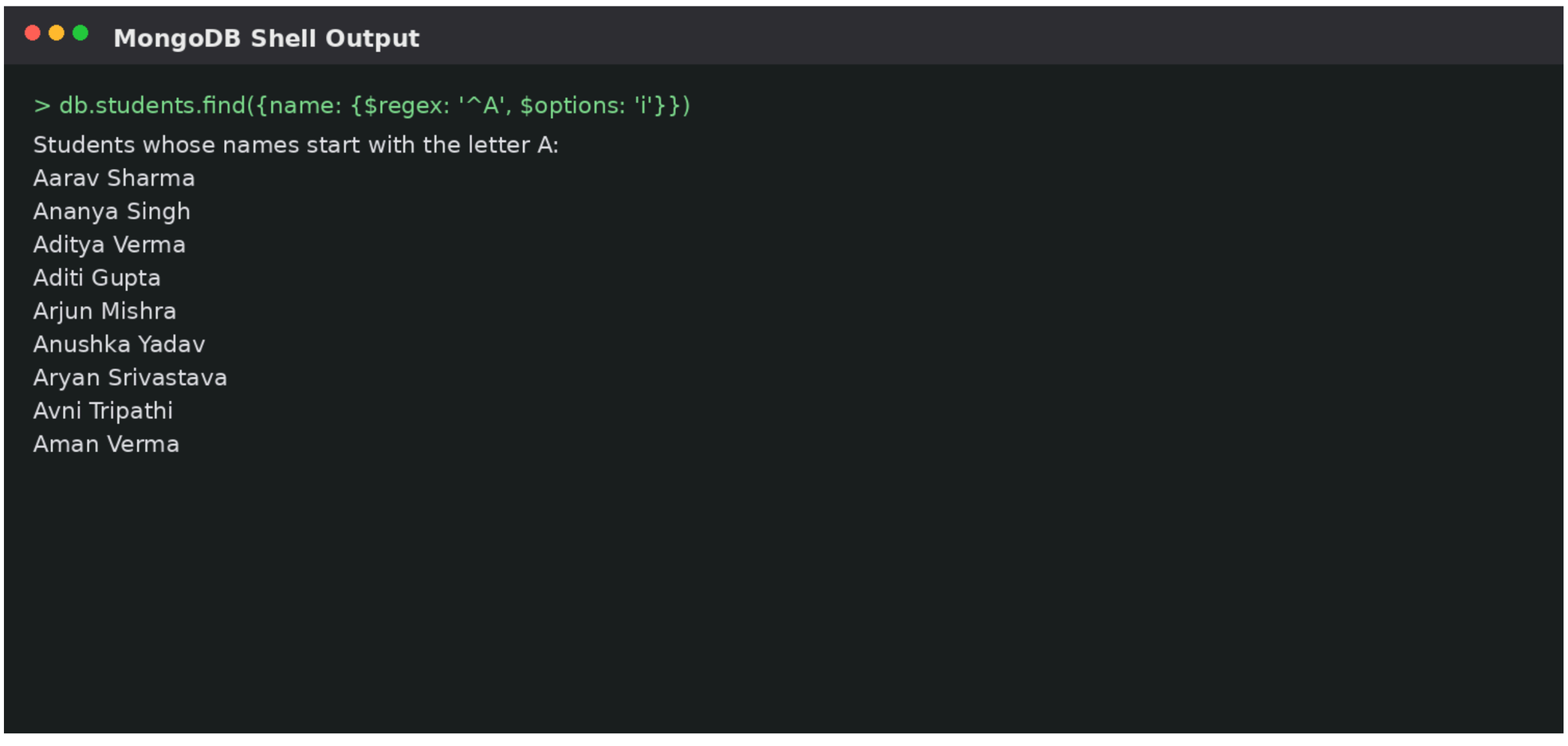
19. REGEX OPERATION

$regex performs pattern matching on text. The pattern ^A finds names beginning with A, while the i option makes the search case-insensitive.

MongoDB Command

```
db.students.find({name: {$regex: '^A', $options: 'i'}})
```

Output Screenshot



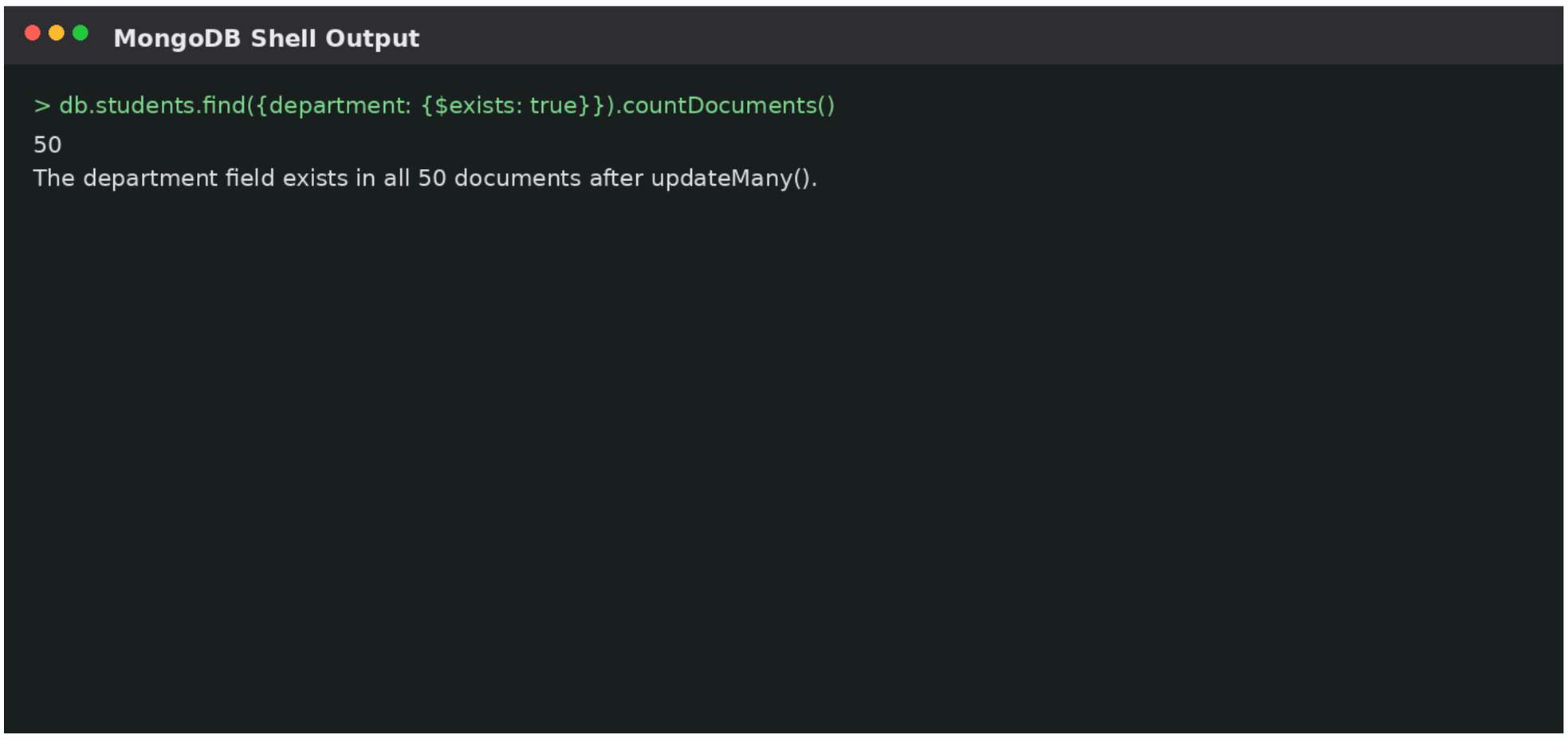
20. EXISTS OPERATOR

$exists checks whether a field is present in a document. After updateMany(), this query confirms that the department field exists in all 50 documents.

MongoDB Command

```
db.students.find({department: {$exists: true}}).countDocuments()
```

Output Screenshot



CONCLUSION

This project demonstrates the basic and intermediate use of MongoDB for managing student information in a NoSQL environment. A collection containing 50 student records was created and populated, after which a wide range of database operations were applied.

The project covered comparison operators such as $gt, $lt, $gte, $lte and $ne; logical operators $and and $or; data presentation operations such as sort, projection and limit; and data management operations such as updateOne(), updateMany() and deleteOne(). distinct(), $regex and $exists were also demonstrated.

Overall, the project provides practical understanding of document-based storage and MongoDB query operations. It shows how a Student Management System can store, search, filter, update and organize student data efficiently using a flexible NoSQL database.